WEEK 2 · LESSON 8 OF 24 · HANDOUT

Prompt Engineering for Developers

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=jmwLzX_ltbQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyData Virginia 2025: Mastering LLMs — from Prompt Engineering to Agentic AI — https://www.youtube.com/watch?v=jmwLzX_ltbQ

🎓 **Go deeper:** Andrej Karpathy: How I use LLMs — https://www.youtube.com/watch?v=EWvNQjAaOHw

Your goal in this lesson: Write prompts that work reliably inside programs — with clear instructions, delimiters, examples and output rules — and measure prompt quality on a small test set instead of guessing.

Prompts inside programs are different

In a chat you can rephrase until you like the answer. In an app the prompt runs **thousands of times** on inputs you never see. It must be **clear, specific and testable**.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
support_messages = [
    ("I sent 20,000 RWF to my brother yesterday but he did not receive it", "payment"),
    ("Internet is very slow in Huye since this morning", "network"),
    ("I forgot my mobile money PIN, please help", "account"),
    ("Money was deducted twice when I paid at the supermarket", "payment"),
    ("No 4G signal in my village for two days", "network"),
    ("My SIM card is blocked after I typed the wrong PIN", "account"),
    ("How do I reverse a payment to a wrong number?", "payment"),
    ("The data bundle I bought is not working", "network"),
]
CATEGORIES = ["payment", "network", "account"]
print(len(support_messages), "messages")
```

Technique 1 — Clear role, task and rules

```
SYSTEM = (
    "You sort customer messages for a mobile-money company in Rwanda. "
    "Reply with exactly one word, one of: payment, network or account. "
    "Do not explain."
)
print(ask("My SIM card is blocked after I typed the wrong PIN", system=SYSTEM))
```

The system prompt states the **role**, the **task**, the **allowed answers** and the **format**.

Technique 2 — Separate instructions from data

Put user text inside clear markers, such as XML-style tags. The model can then tell your instructions from the text it must process — important when that text could contain instructions of its own (Lesson 23):

```
def classify_prompt(message):
    return (
        "Classify the customer message inside <message> tags.\n"
        "Answer with one of: payment, network or account.\n\n"
        f"<message>\n{message}\n</message>"
    )

print(classify_prompt("No 4G signal in my village"))
```

Technique 3 — Show examples (few-shot)

```
FEW_SHOT = (
    "Examples:\n"
    "<message>My airtime purchase failed but money left my account</message> -> payment\n"
    "<message>I cannot open WhatsApp, the internet keeps dropping</message> -> network\n"
    "<message>I want to change the name on my account</message> -> account\n\n"
)
print(ask(FEW_SHOT + classify_prompt("Money was deducted twice at the supermarket"),
system=SYSTEM))
```

Examples teach the format **and** the edge cases ("airtime failed but money left" is a payment problem, not a network one).

Technique 4 — Let the model think first, when it helps

For multi-step problems, ask the model to reason **before** giving the final answer, and put the final answer in a fixed place so code can find it:

```
problem = ("A shop sells 3 kg of beans at 1,200 RWF per kg and 2 kg of rice at 1,500 RWF per
kg. "
           "The customer pays with 10,000 RWF. How much change?")
answer = ask(problem + "\nWork it out step by step, then give the final number on the last line
as ANSWER: <number>.")
print(answer)
```

The correct change is 10,000 − (3,600 + 3,000) = **3,400 RWF**. Many modern models have a built-in "extended thinking" mode that does this reasoning for you.

Technique 5 — Measure, don't guess

A prompt is only "good" if it works on many inputs. Build a small **test set** with the right answers and score each prompt version:

```
def normalise(label):
    label = label.strip().lower().strip(".")
    return label if label in CATEGORIES else "other"

def evaluate(build_prompt, system=SYSTEM):
    correct = 0
    for message, expected in support_messages:
        got = normalise(ask(build_prompt(message), system=system, max_tokens=10))
        correct += got == expected
        if got != expected:
            print(f"  wrong: {message[:45]!r} -> {got} (expected {expected})")
    return correct / len(support_messages)

print("zero-shot accuracy:", evaluate(classify_prompt))
print("few-shot accuracy: ", evaluate(lambda m: FEW_SHOT + classify_prompt(m)))
```

`normalise` protects your app from small surprises ("Payment." or an explanation): anything outside the allowed labels becomes "other", which your code can send to a person. With real data, use at least 50–100 labelled examples, and keep them to re-test whenever you change the prompt or the model.

A prompt checklist

Ask yourself	Example
Is the role clear?	"You sort customer messages for a mobile-money company"
Is the task specific?	"Classify the message inside the tags"
Are the allowed outputs listed?	"one of: payment, network or account"
Is user text separated?	`<message>…</message>`
Are tricky cases shown?	Few-shot examples
Can code check the output?	One word, JSON, or `ANSWER:` line
Has it been tested?	Accuracy on a labelled test set

Key terms

- **Prompt template** — a prompt with slots filled by code.
- **Delimiters** — markers (tags, quotes) separating instructions from data.
- **Few-shot prompting** — giving solved examples in the prompt.
- **Step-by-step reasoning** — asking the model to work through a problem before answering.
- **Prompt evaluation** — scoring a prompt on a labelled test set.

Remember this: In software, a prompt is code: give it a clear role and format, separate data from instructions, show examples, and measure it on a test set before you trust it.

Name: _____ Date: _____

Activity

Task 1

Add five harder messages to the test set (for example "my money was sent but the internet failed"). Which prompt handles them best?

Task 2

Write a prompt that extracts the amount of money from a message and returns only the number.

Task 3

Ask the change problem with and without "step by step". Run each three times. Which is more reliable?

Task 4

Translate the system prompt into Kinyarwanda and test it on Kinyarwanda messages.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why must prompts in apps be more careful than in a chat?
   - A. They run many times on inputs you never see
   - B. Apps cannot use system prompts
   - C. Chats are always wrong
   - D. They cost nothing
2. Why put user text inside tags such as <message>…</message>?
   - A. To separate data from your instructions
   - B. To make it shorter
   - C. To translate it
   - D. To hide it from the model
3. What is few-shot prompting?
   - A. Sending few tokens
   - B. Asking short questions
   - C. Giving solved examples in the prompt
   - D. Using a small model
4. Why ask for the final answer on a fixed line such as ANSWER: <number>?
   - A. It is required by the API
   - B. It reduces tokens to zero
   - C. It looks nicer
   - D. So code can find and check it
5. How do you know if a new prompt is better?
   - A. It scores higher on a labelled test set
   - B. The model says so
   - C. It sounds better
   - D. It is longer
6. What should code do with an output that is not an allowed label?
   - A. Retry forever
   - B. Crash
   - C. Treat it as 'other' and handle it, for example send to a person
   - D. Ignore the customer

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.